Taller U3 — Automatización de BD con Python, Faker y Git

Bases de Datos para Ciencia de Datos – Docente: Miguel Ramos García
Entrega: **Jueves 15 de octubre, 11:59 p.m.** • Nombre: _____

Objetivo

Desarrollar un script en **Python** que automatice la creación y poblado de una base de datos MySQL local utilizando **SQLAlchemy** y la librería **Faker**. Debes gestionar el versionamiento de tu proyecto en un repositorio **público de GitHub**, demostrando buenas prácticas de desarrollo y documentación.

Prerrequisitos

- Python 3.10+ instalado.
- DBeaver.
- Git instalado y configurado.
- Cuenta en GitHub.

Especificaciones del Entregable

Debes generar un documento (PDF) en el cual debe estar el enlace a tu repositorio y una foto de evidencia de la base de datos como se describirá posteriormente. Tu proyecto debe cumplir estrictamente con lo siguiente:

1. Repositorio en GitHub

- Tu repositorio debe ser **PÚBLICO**.
- Debe contener un archivo `.gitignore` configurado (ignorando `.venv`, `__pycache__`, `.env`, investigar gitignore para proyectos python).
- **Importante:** Debe incluir un archivo `.env.example` (que **NO** debe estar en el gitignore) para servir de plantilla de configuración.
- Debe incluir un archivo `requirements.txt` con las librerías necesarias.
- **Historial de Git:** Se requieren **mínimo 5 commits** que evidencien el paso a paso de tu desarrollo (ej: configuración inicial, creación de conexión, lógica de faker, inserción masiva, documentación).

2. Script de Python

- Conexión a tu base de datos local usando **SQLAlchemy**.
- **Creación de estructura:** El script debe crear la tabla automáticamente si no existe.
- **Nombre de la tabla:** Debe seguir el formato `personas_<tu_nombre>` (ej: `personas_miguel`).
- **Columnas:** Debe tener una clave primaria (PK) y atributos coherentes generados por Faker (ej: nombre, correo, fecha_nacimiento, ciudad, etc.). **Mínimo 7 atributos con fakers.**
- **Generación de datos:** Insertar **100.000 (cien mil)** registros utilizando `Faker`.
- **Estructura:** El código debe estar organizado dentro de una función `main()` y utilizar el bloque de ejecución `if __name__ == "__main__":`.

3. Documentación y Evidencia

- **README.md:** Explica qué hace tu código, cómo instalar las dependencias y el comando exacto para ejecutar el script.
- **Evidencia visual:** Pantallazo de pantalla completa (sin recortes) de **DBeaver** ejecutando un `SELECT COUNT(*) ...` sobre la tabla creada, validando los 100.000 registros.

Pistas para el Desarrollo (Lógica Sugerida)

No se provee código. Debes investigar e implementar el siguiente flujo lógico:

1. **Protección de Credenciales (.env vs .env.example):** No escribas tu contraseña de MySQL directamente en el código. Usa la librería `python-dotenv`.
   - El archivo `.env` con tus claves reales va al `.gitignore`.
   - Debes crear y subir un archivo `.env.example` con la estructura de variables que usaste para configurar la conexión (sin las claves reales).
2. **Conexión:** Utiliza `create_engine` de SQLAlchemy. Recuerda que el formato del string de conexión suele ser: `mysql+pymysql://usuario:password@host:puerto/nombre_bd`.
3. **Generación de Datos (Faker):** Crea una instancia de `Faker`. Genera los datos dentro de una lista de diccionarios Python.
4. **Optimización (Opcional):** Aunque puedes insertar los 100.000 registros uno por uno dentro de un ciclo `for`, es una buena práctica profesional investigar cómo pasar una lista completa de diccionarios al método `conn.execute()` (inserción por lotes).

Checklist de Calidad (Auto-revisión)

Antes de enviar tu correo, marca estas casillas para asegurar tu nota máxima:

Git y Repositorio

- ☐ ¿El repositorio es **público**? (Intenta abrir el link en modo incógnito).
- ☐ ¿El archivo `.env` (con tus claves reales) está **ignorado** por git?
- ☐ ¿Subiste el archivo `.env.example` con las variables de configuración?
- ☐ ¿Tienes al menos **5 commits** con mensajes claros?

Script y Base de Datos

- ☐ ¿La tabla se llama `personas_<tu_nombre>`?
- ☐ ¿La tabla tiene al menos **7 atributos** generados con Faker?
- ☐ ¿El script inserta **100.000 registros** correctamente?

Entrega

- ☐ ¿El archivo PDF tiene el nombre `U3_Actividad3_NombreApellido.pdf`?
- ☐ ¿El PDF incluye el link al repo y la captura de DBeaver a **pantalla completa**?

Entregables para calificación

Para realizar la entrega, debes enviar un correo electrónico a **m.ramos@udea.edu.co** cumpliendo los siguientes requisitos:

- **Asunto del correo:** `Actividad 3`
- **Archivo adjunto:** Un único documento en formato **PDF** con el nombre: `U3_Actividad3_NombreApellido.pdf`, el cual debe contener:
  1. **Enlace (Link)** a tu repositorio público de GitHub.

2. **Captura de pantalla completa** mostrando el resultado del query `SELECT COUNT(*) FROM personas_<tu_nombre>;`.

Rúbrica de Evaluación (100 pts)

Componente	Criterios	Puntaje
Repositorio y Git	El repo es público, incluye `.gitignore` correcto, `requirements.txt` con todas las librerías usadas y `.env.example`. Mínimo **5 commits** con mensajes descriptivos.	**25 %**
Script Python y SQLAlchemy	El código conecta correctamente a MySQL, crea la tabla automáticamente y maneja la lógica de conexión. Código limpio y ordenado.	**30 %**
Generación de Datos (Faker)	Uso correcto de la librería Faker. Inserción efectiva de **100.000 registros**. La tabla tiene tu nombre y al menos 7 atributos.	**30 %**
Documentación y Evidencias	`README.md` claro. Pantallazo de DBeaver a **pantalla completa** validando el `COUNT(*)`.	**15 %**

Nota Importante: Si el repositorio es privado y no tengo acceso, la nota será 0. Verifica el enlace en una ventana de incógnito.